

Blok anonimowy

1. Użytkownik podaje dwie liczby X i Y. Następnie pokazujesz mu liczby całkowite od X do Y.
2. Stwórz pętlę pokazującą informację: Liczba 50, Liczba 49...., Liczba 1. Pętla rozpoczyna się od 50 a kończy na 1. Pomiń liczby parzyste pomiędzy 20 a 30.
3. Ustal status klientów:

```
DROP TABLE IF EXISTS Klienci;

CREATE TABLE Klienci (
  Id INT PRIMARY KEY,
  Imie NVARCHAR(50),
  Nazwisko NVARCHAR(50),
  Saldo DECIMAL(10, 2),
  Status NVARCHAR(20)
);

INSERT INTO Klienci (Id, Imie, Nazwisko, Saldo, Status)
VALUES
(1, 'Anna', 'Kowalska', 15000.00, NULL),
(2, 'Jan', 'Nowak', 5000.00, NULL),
(3, 'Ewa', 'Zielińska', 900.00, NULL),
(4, 'Tomasz', 'Wiśniewski', 10500.00, NULL),
(5, 'Katarzyna', 'Wójcik', 0.00, NULL);
```

Napisz skrypt, który:

- Dla każdego klienta zaktualizuje kolumnę Status w zależności od wartości Saldo:
 - VIP jeśli Saldo > 10000
 - Standard jeśli Saldo BETWEEN 1000 AND 10000
 - Zagrożony jeśli Saldo < 1000
 - Zmień dane w tabeli i wypisz je w konsoli poleceniem print

Użyj: CASE, UPDATE oraz jeśli to konieczne - CURSOR

4. Oblicz raty dla zadłużonych klientów (wykorzystaj stworzoną wcześniej tabelę Klienci)

Każdy klient spłaca 10% salda miesięcznie. Symulujemy miesięczne spłaty aż do momentu spłaty całości. Dla każdego miesiąca chcemy wiedzieć:

- IdKlienta
- Miesiąc
- KwotaSpłaty
- PozostałeSaldo

Dane zapisz w tabeli RatySymulacja.

```
DROP TABLE IF EXISTS RatySymulacja;
```

```
CREATE TABLE RatySymulacja (  
    IdKlienta INT,  
    Miesiac INT,  
    KwotaSplaty DECIMAL(10,2),  
    PozostaleSaldo DECIMAL(10,2)  
);
```

5. Napisz program do obliczania kolejnych liczb całkowitych. Np. po podaniu liczby 5 otrzymasz 15(czyli 1+2+3+4+5)
6. Napisz program, który zwróci Ci liczbę liter i cyfr w podanych ciągu znaków i cyfr. Znajdź wcześniej odpowiednią funkcję, która Ci to umożliwi
7. Napisz program w którym użytkownik poda imię i nazwisko szefa, a program zwróci nazwiska jego podwładnych z nazwą departamentu w którym pracują.
8. Napisz kursor, który przechodzi po wszystkich zamówieniach ze statusem Oczekujące.
 - a. Dla każdego zamówienia aktualizuje tabelę logów,
 - i. czy powinno zostać zatwierdzone (Wartosc <= 10000)
 - ii. czy wymaga ręcznej weryfikacji (Wartosc > 10000).
 - b. Użyj: CURSOR, IF, INSERT INTO LogZamowienia

```
DROP TABLE IF EXISTS Zamowienia;
```

```
CREATE TABLE Zamowienia (  
    Id INT PRIMARY KEY,  
    IdKlienta INT,  
    Wartosc DECIMAL(10, 2),  
    Status NVARCHAR(20)  
);
```

```
INSERT INTO Zamowienia (Id, IdKlienta, Wartosc, Status)  
VALUES  
(1, 1, 8000.00, 'Oczekujące'),  
(2, 2, 12000.00, 'Oczekujące'),  
(3, 3, 600.00, 'Oczekujące'),  
(4, 4, 18000.00, 'Oczekujące'),  
(5, 5, 3000.00, 'Oczekujące');
```

Logi:

```
DROP TABLE IF EXISTS LogZamowienia;
```

```
CREATE TABLE LogZamowienia (  
    IdZamowienia INT,
```

```
Decyzja NVARCHAR(50),  
DataWeryfikacji DATETIME DEFAULT GETDATE()  
);
```

9. Zrealizuj scenariusz i wykorzystaj kursor:

- a. Pobieramy Employee_ID, Salary i Department_ID z tabeli EMPLOYEES.
- b. Obliczamy premię jako 10% pensji dla pracowników spoza działu IT (np. DepartmentID != 3) i 15% dla działu IT.
- c. Wstawiamy te wartości do tabeli EMPLOYEE_BONUSES.

Definicja tabeli

```
CREATE TABLE EMPLOYEE_BONUSES (  
    EmployeeID INT PRIMARY KEY,  
    BonusAmount DECIMAL(10,2),  
    CalculationDate DATETIME DEFAULT GETDATE()  
)
```